

Blob Maintenance with Azure Functions

Student Lab: Timer Triggers, HTTP Triggers & Automated Blob Archiving

All commands use Windows Command Prompt (cmd.exe) | Fresh start — no prior lab required

Lab Overview

In this lab you will build a serverless Blob maintenance service using Azure Functions. A Timer Trigger function will automatically move files older than 7 days from an uploads container to an archive container every night at midnight. An HTTP Trigger function gives you an on-demand way to run the same logic — perfect for testing. All Azure resources are created fresh from the command line.

What You Will Build

Lab	Goal
Lab 1: Setup	Sign in to Azure CLI, set variables, create the resource group
Lab 2: Storage	Create a storage account with uploads and archive containers
Lab 3: Key Vault	Store the storage connection string securely in Key Vault
Lab 4: Function App	Scaffold a .NET 8 isolated-worker Function App project
Lab 5: Timer Trigger	Write a function that archives old blobs on a nightly cron schedule
Lab 6: HTTP Trigger	Write an on-demand function that triggers the same archive logic
Lab 7: Test Locally	Run the Function App locally and test both functions with curl
Lab 8: Deploy to Azure	Create a Function App in Azure, connect Key Vault, and deploy
Lab 9: Cleanup	Delete all resources with a single command

Prerequisites — Install These First

- Azure CLI — confirm with: `az --version`
- .NET SDK 8.0 or later — confirm with: `dotnet --version`
- Azure Functions Core Tools v4 — install once with the command below, then confirm with: `func --version`

```
npm install -g azure-functions-core-tools@4 --unsafe-perm true
```

Tip

If npm is not installed, download Node.js from <https://nodejs.org> — the LTS version includes npm. After installing Node.js, reopen cmd.exe and run the npm command above.

Important

Keep ONE cmd.exe window open for the entire lab. Variables set with the set command only last for the current session — if you close the window you must re-run the set commands from Lab 1 using your original SUFFIX value.

Lab 1: CLI Setup

Step 1 — Sign In

1. Run `az login`. A browser window will open — sign in with your Azure account.

```
az login
```

```
mohamedabubakkars@FVF6908RQ05F-mohamedabubakkars ~ % az login
A web browser has been opened at https://login.microsoftonline.com/organizations/oauth2/v2.0/authorize. Please continue the login in the web browser. If no web browser is available or if the web browser fails to open, use device code flow with 'az login --use-device-code'.

Retrieving tenants and subscriptions for the selection...
The following tenants don't contain accessible subscriptions. Use 'az login --allow-no-subscriptions' to have tenant level access.
7f7268a4-7179-4a42-9837-40255b1c21e5 'Presidio Network Solutions'

[Tenant and subscription selection]

No      Subscription name      Subscription ID      Tenant
-----
[1] *   Training-2026      e25db867-f63d-489a-8bce-7de21e5f94a1  Presidio Networked Solutions

The default is marked with an *: the default tenant is 'Presidio Networked Solutions' and subscription is 'Training-2026' (e25db867-f63d-489a-8bce-7de21e5f94a1).

Select a subscription and tenant (Type a number or Enter for no changes): 1

Tenant: Presidio Networked Solutions
Subscription: Training-2026 (e25db867-f63d-489a-8bce-7de21e5f94a1)

[Announcements]
With the new Azure CLI login experience, you can select the subscription you want to use more easily. Learn more about it and its configuration at https://go.microsoft.com/fwlink/?linkid=2271236

If you encounter any problem, please open an issue at https://aka.ms/azclibug

[Warning] The login output has been updated. Please be aware that it no longer displays the full list of available subscriptions by default.
```

Step 2 — Set Variables

2. Type each line exactly. No spaces around the `=` sign.

```
set RG=func-lab-rg
set LOCATION=eastus
set /A SUFFIX=%RANDOM%
set SA_NAME=func-lab-sa-%SUFFIX%
set KV_NAME=func-lab-kv-%SUFFIX%
set FUNC_NAME=func-lab-app-%SUFFIX%
set CONTAINER_UPLOADS=uploads
set CONTAINER_ARCHIVE=archive
```

3. Confirm your values printed correctly:

```
echo %SA_NAME%
echo %KV_NAME%
echo %FUNC_NAME%
```

```

[mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars ~ % echo $RG
abu-function-lab-rg
[mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars ~ % echo $SA_NAME
abufunc488
[mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars ~ % echo $KV_NAME
abu-func-kv-488
[mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars ~ % echo $FUNC_NAME
abu-func-app-488
[mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars ~ % █

```

Write your values here — you will need them if you ever need to restore the variables:

SA_NAME value	abufunc488
----------------------	------------

KV_NAME value	abu-func-kv-488
----------------------	-----------------

FUNC_NAME value	abu-func-app-488
------------------------	------------------

SUFFIX value	488
---------------------	-----

Step 3 — Create the Resource Group

```
az group create --name %RG% --location %LOCATION%
```

```

mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars ~ % az group create \
  --name $RG \
  --location $LOCATION
{
  "id": "/subscriptions/e25db867-f63d-489a-8bce-7de21e5f94a1/resourceGroups/abu-function-lab-rg",
  "location": "eastus",
  "managedBy": null,
  "name": "abu-function-lab-rg",
  "properties": {
    "provisioningState": "Succeeded"
  },
  "tags": null,
  "type": "Microsoft.Resources/resourceGroups"
}
mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars ~ % █

```


Lab 2: Storage Account

The function will move blobs between two containers: uploads (where files land) and archive (where old files go). Both containers live in the same storage account.

Step 1 — Create the Storage Account

```
az storage account create ^
--name %SA_NAME% ^
--resource-group %RG% ^
--location %LOCATION% ^
--sku Standard_LRS ^
--kind StorageV2
```

```
mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars ~ % az storage account create \
--name $SA_NAME \
--resource-group $RG \
--location $LOCATION \
--sku Standard_LRS \
--kind StorageV2
{
  "accessTier": "Hot",
  "accountMigrationInProgress": null,
  "allowBlobPublicAccess": false,
  "allowCrossTenantReplication": false,
  "allowSharedKeyAccess": null,
  "allowSharedKeyAccessForServices": null,
  "allowedCopyScope": null,
  "azureFilesIdentityBasedAuthentication": null,
  "blobRestoreStatus": null,
  "creationTime": "2026-07-02T09:15:07.178525+00:00",
  "customDomain": null,
  "dataCollaborationPolicyProperties": null,
  "defaultToOAuthAuthentication": null,
  "dnsEndpointType": null,
  "dualStackEndpointPreference": null,
  "enableExtendedGroups": null,
  "enableHttpsTrafficOnly": true,
  "enableNfsV3": null,
  "encryption": {
    "encryptionIdentity": null,
    "keySource": "Microsoft.Storage",
    "keyVaultProperties": null,
    "requireInfrastructureEncryption": null,
    "services": {
      "blob": {
        "enabled": true,
        "keyType": "Account",
        "lastEnabledTime": "2026-07-02T09:15:07.626468+00:00"
      }
    }
  },
  "infrastructureEncryption": null,
  "kind": "StorageV2",
  "location": "EastUS",
  "name": "mohamedabubakkars",
  "network": {
    "defaultAction": "Allow",
    "rules": []
  },
  "provisioningState": "Succeeded",
  "resourceGroup": "mohamedabubakkars",
  "sku": "Standard_LRS",
  "tags": {},
  "timeCreated": "2026-07-02T09:15:07.178525+00:00",
  "updateTime": "2026-07-02T09:15:07.178525+00:00"
}
```

Step 2 — Create the Two Containers

```
az storage container create ^
--name %CONTAINER_UPLOADS% ^
--account-name %SA_NAME% ^
--auth-mode login

az storage container create ^
--name %CONTAINER_ARCHIVE% ^
--account-name %SA_NAME% ^
--auth-mode login
```

```
mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars ~ % az storage container create \
--name $CONTAINER_UPLOADS \
--account-name $SA_NAME \
[ --auth-mode login
{
  "created": true
}
mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars ~ % az storage container create \
--name $CONTAINER_ARCHIVE \
--account-name $SA_NAME \
[ --auth-mode login
{
  "created": true
}
mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars ~ % █
```

Step 3 — Capture the Connection String

```
for /f "delims=" %a in ('az storage account show-connection-string --name %SA_NAME%
--resource-group %RG% --query connectionString -o tsv') do set CONN_STRING=%a
```

Tip

Verify it was captured: `echo %CONN_STRING%` — it should start with `DefaultEndpointsProtocol=https`

Lab 3: Key Vault

The connection string will be stored in Key Vault. In Azure, the Function App will read it via a Key Vault Reference — a special app setting syntax that tells Azure to fetch the value from Key Vault automatically, with no code changes needed.

Step 1 — Create the Vault

```
az keyvault create ^
--name %KV_NAME% ^
--resource-group %RG% ^
--location %LOCATION%
```

```
mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars ~ % az keyvault create \
--name %KV_NAME% \
--resource-group %RG% \
--location %LOCATION%
{
  "id": "/subscriptions/e25db867-f63d-489a-8bce-7de21e5f94a1/resourceGroups/abu-function-lab-rg/providers/Microsoft.KeyVault/vaults/abu-func-kv-488",
  "location": "eastus",
  "name": "abu-func-kv-488",
  "properties": {
    "accessPolicies": [],
    "createMode": null,
    "enablePurgeProtection": null,
    "enableRbacAuthorization": true,
    "enableSoftDelete": true,
    "enableForDeployment": false,
    "enableForDiskEncryption": false,
    "enableForTemplateDeployment": false,
    "hsmPoolResourceId": null,
    "networkAcls": null,
    "privateEndpointConnections": null,
    "provisioningState": "Succeeded",
    "publicNetworkAccess": "Enabled",
    "sku": {
      "family": "A",
      "name": "standard"
    },
    "softDeleteRetentionInDays": 90,
    "tenantId": "2a731c61-a2b2-4661-8409-5b861cf40d0c",
    "vaultUri": "https://abu-func-kv-488.vault.azure.net/"
  },
  "resourceGroup": "abu-function-lab-rg",
  "systemData": {
    "createdAt": "2024-07-02T09:21:36.480000+00:00",
    "createdBy": "mabubakkars@presidio.com",
    "createdByType": "User",
    "lastModifiedAt": "2024-07-02T09:21:36.480000+00:00",
    "lastModifiedBy": "mabubakkars@presidio.com",
    "lastModifiedByType": "User"
  },
  "tags": {},
  "type": "Microsoft.KeyVault/vaults"
}
mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars ~ %
```

Step 2 — Grant Yourself Access

4. Capture your Azure AD object ID and set an access policy:

```
for /f "delims=" %a in ('az ad signed-in-user show --query id -o tsv') do set MY_OID=%a

az keyvault set-policy ^
--name %KV_NAME% ^
--object-id %MY_OID% ^
--secret-permissions get set list delete
```

```

mohamedabubakkars@FVF6908RQ05F-mohamedabubakkars ~ % az keyvault set-policy \
--name $KV_NAME \
--object-id $MY_OID \
--secret-permissions get set list delete
{
  "id": "/subscriptions/e25db867-f63d-489a-8bce-7de21e5f94a1/resourceGroups/abu-function-lab-rg/providers/Microsoft.KeyVault/vaults/abu-func-kv-20289",
  "location": "eastus",
  "name": "abu-func-kv-20289",
  "properties": {
    "accessPolicies": [
      {
        "applicationId": null,
        "objectId": "47f0ea90-47ee-46f3-afaf-2ccec0aef784",
        "permissions": {
          "certificates": [
            "all"
          ],
          "keys": [
            "all"
          ],
          "secrets": [
            "set",
            "list",
            "delete",
            "get"
          ],
          "storage": [
            "all"
          ]
        },
        "tenantId": "2a731c61-a2b2-4661-8409-5b861cf40d0c"
      }
    ],
    "createMode": null,
    "enablePurgeProtection": null,
    "enableRbacAuthorization": false,
    "enableSoftDelete": true,
    "enabledForDeployment": false,
    "enabledForDiskEncryption": false,
    "enabledForTemplateDeployment": false,
    "hsmPoolResourceId": null,
  }
}

```

Step 3 — Store the Connection String

```

az keyvault secret set ^
--vault-name %KV_NAME% ^
--name "BlobStorageConnectionString" ^
--value "%CONN_STRING%"

```

```

mohamedabubakkars@FVF6908RQ05F-mohamedabubakkars ~ % az keyvault secret set \
--vault-name $KV_NAME \
--name "BlobStorageConnectionString" \
--value "$CONN_STRING"
{
  "attributes": {
    "created": "2026-07-02T09:40:50+00:00",
    "enabled": true,
    "expires": null,
    "notBefore": null,
    "recoverableDays": 90,
    "recoveryLevel": "Recoverable+Purgeable",
    "updated": "2026-07-02T09:40:50+00:00"
  },
  "contentType": null,
  "id": "https://abu-func-kv-20289.vault.azure.net/secrets/BlobStorageConnectionString/63e07cf3b39e49b8a857bc98889748e0",
  "kid": null,
  "managed": null,
  "name": "BlobStorageConnectionString",
  "tags": {
    "file-encoding": "utf-8"
  },
  "value": "DefaultEndpointsProtocol=https;EndpointSuffix=core.windows.net;AccountName=abufunc488;AccountKey=EUA/GNhEfyGU0mJJGQwH1N0tmvE8YEVie4qWaKcZxw5wAhFu/BvFZMs04qV188vrI2PoN4FHN1Jr+ASTfxAPBw==;BlobEndpoint=https://abufunc488.blob.core.windows.net;/FileEndpoint=https://abufunc488.file.core.windows.net;/QueueEndpoint=https://abufunc488.queue.core.windows.net;/TableEndpoint=https://abufunc488.table.core.windows.net/"
}
mohamedabubakkars@FVF6908RQ05F-mohamedabubakkars ~ %

```

Important

This is the last time the connection string is handled directly. From this point the secret lives only inside Key Vault.

Lab 4: Scaffold the Function App

Step 1 — Create the Project

- Choose a folder on your machine to work in, then `cd` into it.
- Run the following to create the Function App project:

```
func init BlobArchiveFunctions --worker-runtime dotnet-isolated --target-framework net8.0
cd BlobArchiveFunctions
```

Tip

`func init` creates a `.csproj` file, a `Program.cs`, a `host.json`, and a `local.settings.json`. The `dotnet-isolated` model is the modern .NET approach for Azure Functions.

```
mohamedabubakkars@FVF6908RQ05F-mohamedabubakkars AzureLabs % func init BlobArchiveFunctions \
  --worker-runtime dotnet-isolated \
[ --target-framework net8.0

.NET 8 will reach end-of-life on November 10 2026 and will no longer be supported.

Writing /Users/mohamedabubakkars/AzureLabs/BlobArchiveFunctions/.vscode/extensions.json
mohamedabubakkars@FVF6908RQ05F-mohamedabubakkars AzureLabs % code .
mohamedabubakkars@FVF6908RQ05F-mohamedabubakkars AzureLabs % cd BlobArchiveFunctions
mohamedabubakkars@FVF6908RQ05F-mohamedabubakkars BlobArchiveFunctions % █
```

Step 2 — Add the Blob Storage Package

```
dotnet add package Azure.Storage.Blobs
```

```
mohamedabubakkars@FVF6908RQ05F-mohamedabubakkars AzureLabs % cd BlobArchiveFunctions
mohamedabubakkars@FVF6908RQ05F-mohamedabubakkars BlobArchiveFunctions % dotnet add package Azure.Storage.Blobs
info : X.509 certificate chain validation will use the fallback certificate bundle at '/usr/local/share/dotnet/sdk/10.0.203/trustedroots/codesignctl.pem'.
info : X.509 certificate chain validation will use the fallback certificate bundle at '/usr/local/share/dotnet/sdk/10.0.203/trustedroots/timestampctl.pem'.
info : Adding PackageReference for package 'Azure.Storage.Blobs' into project '/Users/mohamedabubakkars/AzureLabs/BlobArchiveFunctions/BlobArchiveFunctions.csproj'.
info : GET https://api.nuget.org/v3/registration5-gz-semver2/azure.storage.blobs/index.json
info : OK https://api.nuget.org/v3/registration5-gz-semver2/azure.storage.blobs/index.json 9310ms
info : Restoring packages for /Users/mohamedabubakkars/AzureLabs/BlobArchiveFunctions/BlobArchiveFunctions.csproj...
info : CACHE https://api.nuget.org/v3/vulnerabilities/index.json
info : CACHE https://api.nuget.org/v3-vulnerabilities/2026.07.01.00.04.29/vulnerability.base.json
info : CACHE https://api.nuget.org/v3-vulnerabilities/2026.07.01.00.04.29/2026.07.02.06.04.35/vulnerability.update.json
info : Package 'Azure.Storage.Blobs' is compatible with all the specified frameworks in project '/Users/mohamedabubakkars/AzureLabs/BlobArchiveFunctions/BlobArchiveFunctions.csproj'.
info : PackageReference for package 'Azure.Storage.Blobs' version '12.29.1' added to file '/Users/mohamedabubakkars/AzureLabs/BlobArchiveFunctions/BlobArchiveFunctions.csproj'.
info : Generating MSBuild file /Users/mohamedabubakkars/AzureLabs/BlobArchiveFunctions/obj/BlobArchiveFunctions.csproj.nuget.g.targets.
info : Writing assets file to disk. Path: /Users/mohamedabubakkars/AzureLabs/BlobArchiveFunctions/obj/project.assets.json
log : Restored /Users/mohamedabubakkars/AzureLabs/BlobArchiveFunctions/BlobArchiveFunctions.csproj (in 249 ms).
mohamedabubakkars@FVF6908RQ05F-mohamedabubakkars BlobArchiveFunctions % █
```


Step 3 — Add the Timer Extension

```
dotnet add package Microsoft.Azure.Functions.Worker.Extensions.Timer
```

Step 4 — Create the Function Files

7. Add the Timer Trigger function file:

```
func new --name NightlyArchive --template "Timer trigger" --language C#
```

8. Add the HTTP Trigger function file:

```
func new --name ManualArchive --template "HTTP trigger" --language C#
```

```
mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars BlobArchiveFunctions % func new --name NightlyArchive --template "Timer trigger"
Template: Timer trigger
Function name: [Timer trigger] NightlyArchive
The function "NightlyArchive" was created successfully from the "Timer trigger" template.
mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars BlobArchiveFunctions % func new --name ManualArchive --template "HTTP trigger"
Template: HTTP trigger
Function name: [HTTP trigger] ManualArchive
The function "ManualArchive" was created successfully from the "HTTP trigger" template.
mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars BlobArchiveFunctions %
```

Lab 5: Write the Timer Trigger Function

The Timer Trigger function runs automatically on a cron schedule. You will replace the generated placeholder code with logic that scans the uploads container and moves files older than 7 days to archive.

Step 1 — Create the Archive Service

9. Create a new file called `BlobArchiveService.cs` in the `BlobArchiveFunctions` folder with the following content:

```
using Azure.Storage.Blobs;
using Microsoft.Extensions.Logging;

namespace BlobArchiveFunctions;

public class BlobArchiveService
{
    private readonly ILogger<BlobArchiveService> _logger;
    private readonly string _connectionString;

    public BlobArchiveService(ILogger<BlobArchiveService> logger, IConfiguration config)
    {
        _logger = logger;
        _connectionString = config["BlobStorageConnectionString"]
            ?? throw new InvalidOperationException("BlobStorageConnectionString is not configured.");
    }

    public async Task<int> ArchiveOldBlobsAsync(int olderThanDays = 7)
    {
        var serviceClient = new BlobServiceClient(_connectionString);
        var uploads = serviceClient.GetBlobContainerClient("uploads");
        var archive = serviceClient.GetBlobContainerClient("archive");

        await archive.CreateIfNotExistsAsync();

        int count = 0;
        var cutoff = DateTimeOffset.UtcNow.AddDays(-olderThanDays);

        await foreach (var blobItem in uploads.GetBlobsAsync())
        {
            if (blobItem.Properties.LastModified < cutoff)
            {
                var sourceBlob = uploads.GetBlobClient(blobItem.Name);
                var destBlob = archive.GetBlobClient(blobItem.Name);

                await destBlob.StartCopyFromUriAsync(sourceBlob.Uri);
                await sourceBlob.DeleteAsync();

                _logger.LogInformation("Archived: {BlobName}", blobItem.Name);
                count++;
            }
        }
    }
}
```



```

        _logger.LogInformation("Archive run complete. Files moved: {Count}", count);
        return count;
    }
}

```

Step 2 — Register the Service in Program.cs

10. Open Program.cs and replace its contents with:

```

using Microsoft.Azure.Functions.Worker;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
using BlobArchiveFunctions;

var host = new HostBuilder()
    .ConfigureFunctionsWorkerDefaults()
    .ConfigureServices(services =>
    {
        services.AddTransient<BlobArchiveService>();
    })
    .Build();

host.Run();

```

Step 3 — Replace the NightlyArchive Function

11. Open NightlyArchive.cs and replace its contents with:

```

using Microsoft.Azure.Functions.Worker;
using Microsoft.Extensions.Logging;

namespace BlobArchiveFunctions;

public class NightlyArchive
{
    private readonly BlobArchiveService _archiveService;
    private readonly ILogger<NightlyArchive> _logger;

    public NightlyArchive(BlobArchiveService archiveService, ILogger<NightlyArchive>
logger)
    {
        _archiveService = archiveService;
        _logger = logger;
    }

    // Runs every day at midnight UTC
    // Cron format: {second} {minute} {hour} {day} {month} {day-of-week}
    [Function("NightlyArchive")]
    public async Task Run([TimerTrigger("0 0 0 * * *")] TimerInfo myTimer)
    {
        _logger.LogInformation("NightlyArchive triggered at: {Time}", DateTime.UtcNow);
        await _archiveService.ArchiveOldBlobsAsync(olderThanDays: 7);
    }
}

```

```
}  
}
```

Tip

The cron expression `0 0 0 * * *` means: at second 0, minute 0, hour 0, every day, every month, every day of the week — i.e. midnight UTC daily.

Lab 6: Write the HTTP Trigger Function

The HTTP Trigger lets you call the same archive logic on demand — useful for testing without waiting for midnight. It also accepts an optional query parameter so you can test with a shorter age window.

Step 1 — Replace the ManualArchive Function

12. Open ManualArchive.cs and replace its contents with:

```
using System.Net;
using Microsoft.Azure.Functions.Worker;
using Microsoft.Azure.Functions.Worker.Http;
using Microsoft.Extensions.Logging;

namespace BlobArchiveFunctions;

public class ManualArchive
{
    private readonly BlobArchiveService _archiveService;
    private readonly ILogger<ManualArchive> _logger;

    public ManualArchive(BlobArchiveService archiveService, ILogger<ManualArchive> logger)
    {
        _archiveService = archiveService;
        _logger = logger;
    }

    [Function("ManualArchive")]
    public async Task<HttpResponseBody> Run(
        [HttpTrigger(AuthorizationLevel.Anonymous, "post")] HttpRequestData req)
    {
        // Optional: pass ?days=1 to archive files older than 1 day (useful for testing)
        var query = System.Web.HttpUtility.ParseQueryString(req.Url.Query);
        int days = int.TryParse(query["days"], out var d) ? d : 7;

        _logger.LogInformation("ManualArchive triggered – archiving files older than {Days} day(s).", days);

        int count = await _archiveService.ArchiveOldBlobsAsync(olderThanDays: days);

        var response = req.CreateResponse(HttpStatusCode.OK);
        await response.WriteStringAsync($"Archive complete. Files moved: {count}");
        return response;
    }
}
```

```
C# BlobArchiveService.cs
📄 host.json
📄 local.settings.json
C# ManualArchive.cs
C# NightlyArchive.cs
C# Program.cs
```

Lab 7: Test Locally

Before deploying to Azure, test the functions on your machine. The local settings file acts as a stand-in for Azure app settings when running locally.

Step 1 — Configure local.settings.json

13. Open local.settings.json (in the BlobArchiveFunctions folder) and replace its contents with your actual connection string:

```
{
  "IsEncrypted": false,
  "Values": {
    "AzureWebJobsStorage": "UseDevelopmentStorage=true",
    "FUNCTIONS_WORKER_RUNTIME": "dotnet-isolated",
    "BlobStorageConnectionString": "PASTE-YOUR-CONNECTION-STRING-HERE"
  }
}
```

Important

Replace PASTE-YOUR-CONNECTION-STRING-HERE with the value from `echo %CONN_STRING%` in your cmd.exe session. local.settings.json is already in .gitignore by default — never commit it to source control.

Step 2 — Build the Project

```
dotnet build
```

```

mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars BlobArchiveFunctions % dotnet build
Restore complete (0.4s)
  WorkerExtensions net8.0 succeeded (1.9s) → obj/Debug/net8.0/WorkerExtensions/bin/Release/net8.0/Microsoft.Azure.Functions.Worker.Extensions.dll
  BlobArchiveFunctions net8.0 succeeded (0.5s) → bin/Debug/net8.0/BlobArchiveFunctions.dll

Build succeeded in 3.5s
mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars BlobArchiveFunctions %

```

Step 3 — Start the Function App

```
func start
```

The Functions runtime will start and print the URLs for both functions. Look for two lines like:

```

NightlyArchive: timerTrigger
ManualArchive: [POST] http://localhost:7071/api/ManualArchive

```

```

mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars BlobArchiveFunctions % dotnet run
Using launch settings from /Users/mohamedabubakkars/AzureLabs/BlobArchiveFunctions/Properties/launchSettings.json...

Azure Functions Core Tools
Core Tools Version:      4.12.1+7f573ec7a43edc895383eb81b78fe86ead2b707b (64-bit)
Function Runtime Version: 4.1048.200.26180

[2026-07-02T10:17:39.391Z] Found /Users/mohamedabubakkars/AzureLabs/BlobArchiveFunctions/BlobArchiveFunctions.csproj. Using for user secrets file configuration.
[2026-07-02T10:17:40.940Z] Worker process started and initialized.

Functions:

    ManualArchive: [POST] http://localhost:7071/api/ManualArchive

    NightlyArchive: timerTrigger

For detailed output, run func with --verbose flag.
[2026-07-02T10:17:49.104Z] Host lock lease acquired by instance ID '00000000000000000000000026A643FC'.

```

Step 4 — Upload a Test File to Blob Storage

14. Open a SECOND cmd.exe window (leave func start running in the first). Upload a test file:

```

az storage blob upload ^
--account-name %SA_NAME% ^
--container-name uploads ^
--name testfile.txt ^
--data "hello from the lab" ^
--auth-mode login

```

Tip

For testing the archive logic without waiting 7 days, you will use the ?days=0 query parameter in the next step — this archives ALL files regardless of age.

Step 5 — Trigger ManualArchive

15. In the second cmd.exe window, call the HTTP trigger with days=0 to archive everything immediately:

```
curl -X POST "http://localhost:7071/api/ManualArchive?days=0"
```

Expected Result

Expected response: Archive complete. Files moved: 1

```
● mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars BlobArchiveFunctions % curl -X POST "http://localhost:7071/api/ManualArchive?days=0"
Archive complete. Files moved: 1
○ mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars BlobArchiveFunctions %
```

Step 6 — Verify the Blob Moved

16. Confirm testfile.txt is now in the archive container and gone from uploads:

```
az storage blob list --account-name %SA_NAME% --container-name archive --auth-mode login
-o table
az storage blob list --account-name %SA_NAME% --container-name uploads --auth-mode login
-o table
```

```
● mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars BlobArchiveFunctions % az storage blob list \
--connection-string "$CONN_STRING" \
--container-name archive \
-o table
```

Name	Blob Type	Blob Tier	Length	Content Type	Last Modified	Snapshot
testfile.txt	BlockBlob	Hot	18	application/octet-stream	2026-07-02T10:20:02+00:00	

```
○ mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars BlobArchiveFunctions %
```


Lab 8: Deploy to Azure

Now deploy the Function App to Azure. The deployed version will read the connection string from Key Vault via a Key Vault Reference — a special app setting syntax that tells Azure to fetch the value automatically. Your function code stays exactly the same.

Step 1 — Create the Function App in Azure

17. Stop func start in the first window (Ctrl+C), then run:

```
az functionapp create ^
--name %FUNC_NAME% ^
--resource-group %RG% ^
--storage-account %SA_NAME% ^
--consumption-plan-location %LOCATION% ^
--runtime dotnet-isolated ^
--functions-version 4 ^
--os-type Windows
```

```
mohamedabubakkars@FVF6908RQ05F-mohamedabubakkars BlobArchiveFunctions % az functionapp create \
--name $FUNC_NAME \
--resource-group $RG \
--storage-account $SA_NAME \
--consumption-plan-location $LOCATION \
--runtime dotnet-isolated \
--functions-version 4 \
--os-type Windows
Error while trying to create and configure an Application Insights for the Function App. Please use the Azure Portal to create and configure the Application
Insights, if needed.
{
  "autoGeneratedDomainNameLabelScope": null,
  "availabilityState": "Normal",
  "clientAffinityEnabled": false,
  "clientAffinityPartitioningEnabled": null,
  "clientAffinityProxyEnabled": false,
  "clientCertEnabled": false,
  "clientCertExclusionPaths": null,
  "clientCertMode": "Required",
  "cloningInfo": null,
  "containerSize": 1536,
  "customDomainVerificationId": "46807E15E4B3288A1BCC91EA4477FF9CF6E3CE676FACDEA0663A4E096E2CCC81",
  "dailyMemoryTimeQuota": 0,
  "daprConfig": null,
  "defaultHostName": "abu-func-app-488.azurewebsites.net",
  "dnsConfiguration": {
    "dnsAltServer": null,
    "dnsLegacySortOrder": null,
    "dnsMaxCacheTimeout": null,
    "dnsRetryAttemptCount": null,
    "dnsRetryAttemptTimeout": null,
    "dnsServers": null
  },
  "enabled": true,
  "enabledHostNames": [
    "abu-func-app-488.azurewebsites.net",
    "abu-func-app-488.scm.azurewebsites.net"
  ],
  "endToEndEncryptionEnabled": false,
```

Function App URL
(https://FUNC_NAME.azurewebsites.net)

abu-func-app-488.azurewebsites.net

Step 2 — Assign a Managed Identity

```
az functionapp identity assign --name %FUNC_NAME% --resource-group %RG%
```

18. Capture the identity's principal ID:

```
for /f "delims=" %a in ('az functionapp identity show --name %FUNC_NAME% --resource-group %RG% --query principalId -o tsv') do set PRINCIPAL_ID=%a
```

```
mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars BlobArchiveFunctions % az functionapp show \
--name $FUNC_NAME \
--resource-group $RG \
--query "{Name:name,State:state,Host:defaultHostName}" \
-o table
Name                State      Host
-----
abu-func-app-488    Running   abu-func-app-488.azurewebsites.net
mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars BlobArchiveFunctions % az functionapp identity assign \
--name $FUNC_NAME \
--resource-group $RG
{
  "principalId": "2a9ff4ad-ee79-4371-91e2-2b598d970347",
  "tenantId": "2a731c61-a2b2-4661-8409-5b861cf40d0c",
  "type": "SystemAssigned",
  "userAssignedIdentities": null
}
mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars BlobArchiveFunctions %
```

Step 3 — Grant the Identity Key Vault Access

```
az keyvault set-policy ^
--name %KV_NAME% ^
--object-id %PRINCIPAL_ID% ^
--secret-permissions get list
```

```
mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars BlobArchiveFunctions % az keyvault set-policy \
--name $KV_NAME \
--object-id $PRINCIPAL_ID \
--secret-permissions get list
{
  "id": "/subscriptions/e25db867-f63d-489a-8bce-7de21e5f94a1/resourceGroups/abu-function-lab-rg/providers/Microsoft.KeyVault/vaults/abu-func-kv-20289",
  "location": "eastus",
  "name": "abu-func-kv-20289",
  "properties": {
    "accessPolicies": [
      {
        "applicationId": null,
        "objectId": "47f0ea90-47ee-46f3-afaf-2cc0c0aef784",
        "permissions": {
          "certificates": [
            "all"
          ],
          "keys": [
            "all"
          ],
          "secrets": [
            "set",
            "list",
            "delete",
            "get"
          ],
          "storage": [
            "all"
          ]
        }
      }
    ]
  }
}
```

Step 4 — Set the Key Vault Reference App Setting

Instead of pasting the connection string directly, use a Key Vault Reference. Azure resolves the `@Microsoft.KeyVault(...)` syntax automatically and injects the real value at runtime:

```
az functionapp config appsettings set ^
--name %FUNC_NAME% ^
--resource-group %RG% ^
--settings
"BlobStorageConnectionString=@Microsoft.KeyVault(VaultName=%KV_NAME%;SecretName=BlobStorageConnectionString"
```

Tip

This is Key Vault Reference syntax. Your function code reads BlobStorageConnectionString exactly as before — Azure quietly swaps in the real value from Key Vault before handing it to your code. No code changes required.

```
mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars BlobArchiveFunctions % az functionapp config appsettings set \
--name $FUNC_NAME \
--resource-group $RG \
--settings BlobStorageConnectionString="@Microsoft.KeyVault(SecretUri=https://$KV_NAME.vault.azure.net/secrets/BlobStorageConnectionString)"
[
{
  "name": "FUNCTIONS_WORKER_RUNTIME",
  "slotSetting": false,
  "value": null
},
{
  "name": "WEBSITE_USE_PLACEHOLDER_DOTNETISOLATED",
  "slotSetting": false,
  "value": null
},
{
  "name": "FUNCTIONS_EXTENSION_VERSION",
  "slotSetting": false,
  "value": null
},
{
  "name": "AzureWebJobsStorage",
  "slotSetting": false,
  "value": null
},
{
  "name": "WEBSITE_CONTENTAZUREFILECONNECTIONSTRING",
  "slotSetting": false,
  "value": null
},
{
  "name": "WEBSITE_CONTENTSHARE",
  "slotSetting": false,
  "value": null
},
{

```

Step 5 — Publish and Deploy

```
func azure functionapp publish %FUNC_NAME%
```

```

mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars BlobArchiveFunctions % func azure functionapp publish $FUNC_NAME
Detected target framework: net8.0
Setting functions site property 'netFrameworkVersion' to 'v8.0'
Determining projects to restore...
All projects are up-to-date for restore.
Determining projects to restore...
Restored /Users/mohamedabubakkars/AzureLabs/BlobArchiveFunctions/obj/release/net8.0/WorkerExtensions/WorkerExtensions.csproj (in 9.55 sec).
WorkerExtensions -> /Users/mohamedabubakkars/AzureLabs/BlobArchiveFunctions/obj/release/net8.0/WorkerExtensions/bin/Release/net8.0/Microsoft.Azure.Fu
s.Worker.Extensions.dll
BlobArchiveFunctions -> /Users/mohamedabubakkars/AzureLabs/BlobArchiveFunctions/bin/publish/BlobArchiveFunctions.dll

Build succeeded.
    0 Warning(s)
    0 Error(s)

Time Elapsed 00:00:17.72

Getting site publishing info...
[2026-07-02T18:47:31.591Z] Starting the function app deployment...
Creating archive for current directory...
Uploading 4.09 MB [#####]
Upload completed successfully.
Deployment completed successfully.
[2026-07-02T18:48:00.881Z] Syncing triggers...
Functions in abu-func-app-488:
    ManualArchive - [httpTrigger]
        Invoke url: https://abu-func-app-488.azurewebsites.net/api/manualarchive
    NightlyArchive - [timerTrigger]
mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars BlobArchiveFunctions %

```

Step 6 — Test the Deployed HTTP Trigger

19. Upload a test file to blob storage first:

```

az storage blob upload ^
--account-name %SA_NAME% ^
--container-name uploads ^
--name cloudtest.txt ^
--data "uploaded for cloud test" ^
--auth-mode login

```

20. Trigger the ManualArchive function in the cloud:

```

curl -X POST "https://%FUNC_NAME%.azurewebsites.net/api/ManualArchive?days=0"

```

Expected Result

Expected response: Archive complete. Files moved: 1

```

mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars BlobArchiveFunctions % curl -X POST "https://$FUNC_NAME.azurewebsites.net/api/ManualArchive?days=0"
Archive complete. Files moved: 1
mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars BlobArchiveFunctions %

```

21. Verify the blob moved to archive:

```

az storage blob list --account-name %SA_NAME% --container-name archive --auth-mode login
-o table

```

```
mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars BlobArchiveFunctions % az storage blob list \
--connection-string "$CONN_STRING" \
--container-name archive \
-o table
```

Name	Blob Type	Blob Tier	Length	Content Type	Last Modified	Snapshot
cloudtest.txt	BlockBlob	Hot	23	application/octet-stream	2026-07-02T10:51:50+00:00	
testfile.txt	BlockBlob	Hot	18	application/octet-stream	2026-07-02T10:20:02+00:00	

```
mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars BlobArchiveFunctions %
```

Lab 9: Cleanup

Important

This permanently deletes all resources. Only run when you are completely finished with the lab.

```
az group delete --name %RG% --yes --no-wait
```

```
mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars BlobArchiveFunctions % az group delete --name $RG --yes --no-wait
mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars BlobArchiveFunctions %
```

Reflection Questions

- What is the difference between a Timer Trigger and an HTTP Trigger? When would you choose one over the other?
 - Timer Trigger** runs automatically based on a schedule (cron expression). It is used for scheduled tasks like backups, cleanup, and report generation.
 - HTTP Trigger** runs when an HTTP request is received. It is used for APIs, webhooks, and user-initiated operations.
- What does the cron expression `0 0 0 * * *` mean? How would you change it to run every Monday at 6:00 AM?
 - `0 0 0 * * *` means the function runs every day at **12:00 AM (midnight UTC)**.
 - Every Monday at **6:00 AM UTC**:
- What is a Key Vault Reference? How is it different from storing the connection string directly as an app setting?

A **Key Vault Reference** allows Azure App Settings to securely retrieve secrets directly from Azure Key Vault at runtime.

4. The NightlyArchive function runs at midnight UTC. If your business is in India (IST = UTC+5:30), what cron expression would run it at midnight IST?

Midnight IST is **6:30 PM UTC (previous day)**.

Cron expression: 0 30 18 * * *

This runs every day at **18:30 UTC**, which corresponds to **12:00 AM IST** the next day.

Common Errors

Error	Fix
func: command not found	Azure Functions Core Tools not installed. Run: <code>npm install -g azure-functions-core-tools@4 --unsafe-perm true</code>
Variable not set (%SA_NAME% prints literally)	Session was closed. Re-run all set commands from Lab 1 using your saved SUFFIX value
BlobStorageConnectionString is not configured	Check local.settings.json — the value must be the actual connection string, not a placeholder
403 on Key Vault (deployed)	Managed Identity access policy not set or wrong principalId — re-run <code>az keyvault set-policy</code> with the correct PRINCIPAL_ID
Key Vault Reference shows as literal string	The Function App identity was not granted get permission on Key Vault secrets — check Step 3 of Lab 8
func start fails — port in use	Another process is using port 7071. Close it or run: <code>func start --port 7072</code>
dotnet build errors on the function files	Ensure the <code>func new</code> command ran successfully and both .cs files exist in the project folder
Blob not archived — count is 0	The file is newer than the days threshold. Test with <code>?days=0</code> to archive all files regardless of age